

Pack 11 — Connector Catalog + Device Policy Enforcement (real code)

RemoteDesk · Pack 11 — Connector Catalog + Device Policy Enforcement (real code)

Codex / Claude Code handoff. Copy each file to the path in its heading. Build order: shared → api → web → desktop → tests. Every file is tagged **SAFE_DIRECT_COPY** (new file, drop in) or **REVIEW_REQUIRED** (patch to an existing file — merge by hand).

Scope: a read-only connector catalog (Slack, Teams, Jira, Zendesk, S3, SIEM webhook) with install/uninstall placeholders + audit logging, and desktop device-policy enforcement that fails **closed** for remote input / clipboard / file transfer when the API is unreachable. **No OAuth secrets, no real provider calls.**

Zip-ready file tree

```
remotedesk/
  packages/shared/src/
    connectors/
      types.ts                # SAFE_DIRECT_COPY
      catalog.ts              # SAFE_DIRECT_COPY
      safety.ts               # SAFE_DIRECT_COPY
      schema.ts               # SAFE_DIRECT_COPY
      index.ts                # SAFE_DIRECT_COPY
    policy/
      devicePolicy.ts         # SAFE_DIRECT_COPY
      index.ts                # SAFE_DIRECT_COPY
    errors/errorCodes.ts     # REVIEW_REQUIRED (patch)
    index.ts                  # REVIEW_REQUIRED (patch)
  apps/api/
    prisma/schema.prisma     # REVIEW_REQUIRED (patch)
    prisma/migrations/0011_connector_catalog/migration.sql # SAFE_DIRECT_COPY
    src/lib/connectorCatalog.ts # SAFE_DIRECT_COPY
    src/repositories/connectorRepository.ts # SAFE_DIRECT_COPY
    src/services/connector/connector.service.ts # SAFE_DIRECT_COPY
    src/services/connector/connectorAudit.ts # SAFE_DIRECT_COPY
    src/routes/connector.routes.ts # SAFE_DIRECT_COPY
    src/db/seed/connectors.seed.ts # SAFE_DIRECT_COPY
    src/server.ts             # REVIEW_REQUIRED (patch)
  apps/web/app/dashboard/
    connectors/page.tsx       # SAFE_DIRECT_COPY
    connectors/connectorApi.ts # SAFE_DIRECT_COPY
    connectors/ConnectorCard.tsx # SAFE_DIRECT_COPY
    connectors/connectors.module.css # SAFE_DIRECT_COPY
    page.tsx                  # REVIEW_REQUIRED (patch — add link)
  apps/desktop/src/renderer/src/
    services/devicePolicy.ts  # SAFE_DIRECT_COPY
    services/devicePolicyPoller.ts # SAFE_DIRECT_COPY
    services/devicePolicyGuards.ts # SAFE_DIRECT_COPY
```

```

    session/sessionSetup.ts          # REVIEW_REQUIRED (patch)
tests/
    connectors/catalog.test.ts       # SAFE_DIRECT_COPY
    connectors/safety.test.ts        # SAFE_DIRECT_COPY
    device-policy/policy-enforcement.test.ts # SAFE_DIRECT_COPY
docs/
    connector-catalog.md             # SAFE_DIRECT_COPY
    device-policy-enforcement.md     # SAFE_DIRECT_COPY
generated-remotedesk-pack11-merge-summary.md # doNotMerge (handoff meta)
generated-remotedesk-pack11-code-review.md   # doNotMerge (handoff meta)
generated-remotedesk-pack11-manifest.json    # doNotMerge (handoff meta)

```

Part A — Shared contracts

`packages/shared/src/connectors/types.ts` — `SAFE_DIRECT_COPY`

```

/** Connector taxonomy shared by API seed, API service, web catalog UI, and tests. */
export type ConnectorCategory = 'chat' | 'ticketing' | 'storage' | 'security';

/** Catalog-level availability authored in code (not per-org). */
export type ConnectorAvailability = 'available' | 'coming_soon';

/** Per-org, per-connector display status returned to the UI. */
export type ConnectorStatus = 'available' | 'installed' | 'coming_soon';

/** A capability a connector exposes. Display-only in Pack 11 (no execution). */
export type ConnectorCapability =
  | 'notify'
  | 'create_ticket'
  | 'sync_tickets'
  | 'export_logs'
  | 'archive_recordings';

/** Canonical, code-authored connector definition. No secrets, no endpoints. */
export interface ConnectorDefinition {
  /** Stable slug, e.g. "slack". Lowercase, used as the primary key everywhere. */
  key: string;
  name: string;
  category: ConnectorCategory;
  description: string;
  availability: ConnectorAvailability;
  capabilities: ConnectorCapability[];
  /** Marketing/docs link only — never an OAuth or API endpoint. */
  docsUrl?: string;
}

/** What the catalog endpoint returns: definition + this org's install state. */
export interface ConnectorCatalogEntry extends ConnectorDefinition {
  installed: boolean;
  installedAt: string | null;
  /** Effective status for the card: coming_soon wins over installed/available. */
  status: ConnectorStatus;
}

```

```

/** Audit event kinds emitted by install/uninstall. */
export type ConnectorAuditAction = 'connector.installed' | 'connector.uninstalled';

export interface ConnectorAuditEventDto {
  id: string;
  orgId: string;
  connectorKey: string;
  action: ConnectorAuditAction;
  actorUserId: string;
  createdAt: string;
}

```

packages/shared/src/connectors/catalog.ts — SAFE_DIRECT_COPY

```

import type { ConnectorDefinition } from './types';

/**
 * The canonical default connector catalog. This is the single source of truth:
 * the API seed writes these rows, the API service falls back to them, and tests
 * assert against them. Editing a connector here is the only place it changes.
 */
export const DEFAULT_CONNECTORS: readonly ConnectorDefinition[] = [
  {
    key: 'slack',
    name: 'Slack',
    category: 'chat',
    description: 'Post session and incident notifications to a Slack channel.',
    availability: 'available',
    capabilities: ['notify'],
    docsUrl: 'https://docs.remotedesk.example/connectors/slack',
  },
  {
    key: 'teams',
    name: 'Microsoft Teams',
    category: 'chat',
    description: 'Send session alerts and approvals to a Teams channel.',
    availability: 'available',
    capabilities: ['notify'],
    docsUrl: 'https://docs.remotedesk.example/connectors/teams',
  },
  {
    key: 'jira',
    name: 'Jira',
    category: 'ticketing',
    description: 'Open and link Jira issues from remote support sessions.',
    availability: 'available',
    capabilities: ['create_ticket', 'sync_tickets'],
    docsUrl: 'https://docs.remotedesk.example/connectors/jira',
  },
  {
    key: 'zendesk',
    name: 'Zendesk',
    category: 'ticketing',
    description: 'Attach session summaries to Zendesk tickets.',
    availability: 'available',
    capabilities: ['create_ticket', 'sync_tickets'],
    docsUrl: 'https://docs.remotedesk.example/connectors/zendesk',
  },

```

```

    },
    {
      key: 's3',
      name: 'Amazon S3',
      category: 'storage',
      description: 'Archive session recordings and exports to an S3 bucket.',
      availability: 'available',
      capabilities: ['archive_recordings'],
      docsUrl: 'https://docs.remotedesk.example/connectors/s3',
    },
    {
      key: 'siem-webhook',
      name: 'SIEM Webhook',
      category: 'security',
      description: 'Stream audit events to a generic SIEM via outbound webhook.',
      availability: 'coming_soon',
      capabilities: ['export_logs'],
      docsUrl: 'https://docs.remotedesk.example/connectors/siem-webhook',
    },
  ],
];

/** 0(1) lookup by key, derived from the list above. */
export const CONNECTORS_BY_KEY: Readonly<Record<string, ConnectorDefinition>> =
  Object.freeze(
    Object.fromEntries(DEFAULT_CONNECTORS.map((c) => [c.key, c])),
  );

export function getConnectorDefinition(key: string): ConnectorDefinition | undefined {
  return CONNECTORS_BY_KEY[key];
}

```

packages/shared/src/connectors/safety.ts — SAFE_DIRECT_COPY

```

import type { ConnectorDefinition, ConnectorStatus } from './types';
import { getConnectorDefinition } from './catalog';

/**
 * Pure safety guards shared by server and client so install rules never drift.
 * Pack 11 has no OAuth/secrets — these guards enforce that invariant.
 */

/** Field names that must never appear on a connector definition payload. */
export const FORBIDDEN_SECRET_KEYS: readonly string[] = [
  'clientSecret',
  'client_secret',
  'apiKey',
  'api_key',
  'token',
  'accessToken',
  'refreshToken',
  'password',
  'privateKey',
];

/** Throws if a (possibly untrusted) object carries secret-looking fields. */
export function assertNoSecretFields(obj: Record<string, unknown>): void {
  for (const key of Object.keys(obj)) {

```

```

    if (FORBIDDEN_SECRET_KEYS.includes(key)) {
      throw new Error(`Connector payload must not contain secret field "${key}"`);
    }
  }
}

/** A connector can be installed only if it exists and is not coming_soon. */
export function isInstallable(key: string): boolean {
  const def = getConnectorDefinition(key);
  return !!def && def.availability !== 'coming_soon';
}

/** Compute the effective UI status from availability + install state. */
export function resolveStatus(
  def: Pick<ConnectorDefinition, 'availability'>,
  installed: boolean,
): ConnectorStatus {
  if (def.availability === 'coming_soon') return 'coming_soon';
  return installed ? 'installed' : 'available';
}

/** Normalize/validate a connector key into its canonical slug, or null. */
export function normalizeConnectorKey(raw: string): string | null {
  const key = raw.trim().toLowerCase();
  return getConnectorDefinition(key) ? key : null;
}

```

packages/shared/src/connectors/schema.ts — SAFE_DIRECT_COPY

```

import { z } from 'zod';
import { DEFAULT_CONNECTORS } from './catalog';

const KEYS = DEFAULT_CONNECTORS.map((c) => c.key) as [string, ...string[]];

/** Route param/body validation for install + uninstall. */
export const connectorKeyParamSchema = z.object({
  key: z.enum(KEYS),
});

export const installConnectorSchema = z.object({
  /** Optional free-form display label; never secrets. Capped to keep audit clean. */
  label: z.string().min(1).max(120).optional(),
});

export type ConnectorKeyParam = z.infer<typeof connectorKeyParamSchema>;
export type InstallConnectorInput = z.infer<typeof installConnectorSchema>;

```

packages/shared/src/connectors/index.ts — SAFE_DIRECT_COPY

```

export * from './types';
export * from './catalog';
export * from './safety';
export * from './schema';

```

packages/shared/src/policy/devicePolicy.ts — SAFE_DIRECT_COPY

```

/**
 * Device policy contract shared by the API (source of truth) and the desktop

```

```

* client (enforcer). Enforcement fails CLOSED: when policy is unknown, every
* sensitive capability is denied.
*/
export interface DevicePolicy {
  deviceId: string;
  /** Operator may send mouse/keyboard to this device. */
  allowRemoteInput: boolean;
  /** Clipboard sync (copy/paste) permitted. */
  allowClipboard: boolean;
  /** File transfer to/from this device permitted. */
  allowFileTransfer: boolean;
  /** Remote audio capture permitted. */
  allowAudio: boolean;
  /** Hard session cap in minutes; null = no cap. */
  maxSessionMinutes: number | null;
  /** Monotonic version for staleness checks. */
  version: number;
  updatedAt: string;
}

/** The capabilities the desktop session gates on. */
export type PolicyCapability =
  | 'remoteInput'
  | 'clipboard'
  | 'fileTransfer'
  | 'audio';

/**
 * The fail-closed policy. Used whenever the API is unreachable, the payload is
 * malformed, or no policy has ever been fetched. Everything denied.
 */
export const FAIL_CLOSED_POLICY: Omit<DevicePolicy, 'deviceId' | 'updatedAt'> = {
  allowRemoteInput: false,
  allowClipboard: false,
  allowFileTransfer: false,
  allowAudio: false,
  maxSessionMinutes: 0,
  version: 0,
};

/** Build a concrete fail-closed policy for a device. */
export function failClosedPolicy(deviceId: string): DevicePolicy {
  return {
    deviceId,
    ...FAIL_CLOSED_POLICY,
    updatedAt: new Date(0).toISOString(),
  };
}

/** Map a capability to its policy flag. Pure, used by client + tests. */
export function isCapabilityAllowed(
  policy: DevicePolicy,
  capability: PolicyCapability,
): boolean {
  switch (capability) {
    case 'remoteInput':
      return policy.allowRemoteInput;

```

```

    case 'clipboard':
      return policy.allowClipboard;
    case 'fileTransfer':
      return policy.allowFileTransfer;
    case 'audio':
      return policy.allowAudio;
    default:
      return false;
  }
}

/** Runtime guard: is this an acceptably-shaped DevicePolicy? */
export function isDevicePolicy(value: unknown): value is DevicePolicy {
  if (!value || typeof value !== 'object') return false;
  const p = value as Record<string, unknown>;
  return (
    typeof p.deviceId === 'string' &&
    typeof p.allowRemoteInput === 'boolean' &&
    typeof p.allowClipboard === 'boolean' &&
    typeof p.allowFileTransfer === 'boolean' &&
    typeof p.allowAudio === 'boolean' &&
    typeof p.version === 'number'
  );
}

```

packages/shared/src/policy/index.ts — SAFE_DIRECT_COPY

```
export * from './devicePolicy';
```

packages/shared/src/errors/errorCodes.ts — REVIEW_REQUIRED (patch)

Add the connector codes to the existing `ErrorCodes` const. Keep all existing entries.

```

export const ErrorCodes = {
  // ...existing codes...
  CONNECTOR_NOT_FOUND: 'CONNECTOR_NOT_FOUND',
  CONNECTOR_NOT_INSTALLABLE: 'CONNECTOR_NOT_INSTALLABLE',
  CONNECTOR_ALREADY_INSTALLED: 'CONNECTOR_ALREADY_INSTALLED',
  CONNECTOR_NOT_INSTALLED: 'CONNECTOR_NOT_INSTALLED',
} as const;

```

packages/shared/src/index.ts — REVIEW_REQUIRED (patch)

Add these two barrels next to the existing exports.

```

// ...existing exports...
export * from './connectors';
export * from './policy';

```

Part B — API: Prisma schema + migration

apps/api/prisma/schema.prisma — REVIEW_REQUIRED (patch)

Append these three models. They reference the existing `Organization` model by id. Run `prisma generate` after merging.

```

model ConnectorDefinition {
  key          String   @id

```

```

    name          String
    category       String
    description     String
    availability    String    // "available" | "coming_soon"
    capabilities    String[]
    docsUrl        String?
    createdAt      DateTime @default(now())
    updatedAt      DateTime @updatedAt

    installations ConnectorInstallation[]

    @@index([category])
}

model ConnectorInstallation {
    id          String @id @default(cuid())
    orgId       String
    connectorKey String
    label       String?
    installedById String
    installedAt  DateTime @default(now())

    connector ConnectorDefinition @relation(fields: [connectorKey], references: [key], onDelete: Cascade)

    @@unique([orgId, connectorKey])
    @@index([orgId])
    @@index([connectorKey])
}

model ConnectorAuditEvent {
    id          String @id @default(cuid())
    orgId       String
    connectorKey String
    action       String    // "connector.installed" | "connector.uninstalled"
    actorUserId  String
    createdAt    DateTime @default(now())

    @@index([orgId])
    @@index([connectorKey])
    @@index([orgId, createdAt])
}

```

[apps/api/prisma/migrations/0011_connector_catalog/migration.sql](#) — **SAFE_DIRECT_COPY**

```

-- ConnectorDefinition
CREATE TABLE "ConnectorDefinition" (
    "key"          TEXT PRIMARY KEY,
    "name"         TEXT NOT NULL,
    "category"     TEXT NOT NULL,
    "description"  TEXT NOT NULL,
    "availability" TEXT NOT NULL,
    "capabilities" TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[],
    "docsUrl"      TEXT,
    "createdAt"    TIMESTAMP NOT NULL DEFAULT now(),
    "updatedAt"    TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "ConnectorDefinition_category_idx" ON "ConnectorDefinition"("category");

```



```

-- ConnectorInstallation
CREATE TABLE "ConnectorInstallation" (
  "id"          TEXT PRIMARY KEY,
  "orgId"       TEXT NOT NULL,
  "connectorKey" TEXT NOT NULL REFERENCES "ConnectorDefinition"("key") ON DELETE CASCADE,
  "label"       TEXT,
  "installedById" TEXT NOT NULL,
  "installedAt"  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX "ConnectorInstallation_orgId_connectorKey_key" ON "ConnectorInstallation"("orgId", "connectorKey");
CREATE INDEX "ConnectorInstallation_orgId_idx" ON "ConnectorInstallation"("orgId");
CREATE INDEX "ConnectorInstallation_connectorKey_idx" ON "ConnectorInstallation"("connectorKey");

-- ConnectorAuditEvent
CREATE TABLE "ConnectorAuditEvent" (
  "id"          TEXT PRIMARY KEY,
  "orgId"       TEXT NOT NULL,
  "connectorKey" TEXT NOT NULL,
  "action"      TEXT NOT NULL,
  "actorUserId" TEXT NOT NULL,
  "createdAt"   TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "ConnectorAuditEvent_orgId_idx" ON "ConnectorAuditEvent"("orgId");
CREATE INDEX "ConnectorAuditEvent_connectorKey_idx" ON "ConnectorAuditEvent"("connectorKey");
CREATE INDEX "ConnectorAuditEvent_orgId_createdAt_idx" ON "ConnectorAuditEvent"("orgId", "createdAt");

```

Part C — API: repository, services, lib, routes, seed

`apps/api/src/lib/connectorCatalog.ts` — `SAFE_DIRECT_COPY`

```

import {
  DEFAULT_CONNECTORS,
  getConnectorDefinition,
  resolveStatus,
  type ConnectorCatalogEntry,
  type ConnectorDefinition,
} from '@remotedesk/shared';

/**
 * Pure catalog assembly: merge the code-authored definitions with an org's
 * installation map into the read-only catalog the API serves. No DB access here
 * so it stays trivially testable.
 */
export function buildCatalog(
  installed: ReadonlyMap<string, { installedAt: Date }>,
): ConnectorCatalogEntry[] {
  return DEFAULT_CONNECTORS.map((def) => toEntry(def, installed.get(def.key)));
}

export function toEntry(
  def: ConnectorDefinition,
  install?: { installedAt: Date },
): ConnectorCatalogEntry {

```

```

const isInstalled = !!install;
return {
  ...def,
  installed: isInstalled,
  installedAt: install ? install.installedAt.toISOString() : null,
  status: resolveStatus(def, isInstalled),
};
}

/** Re-export for callers that only need a single definition. */
export { getConnectorDefinition };

```

apps/api/src/repositories/connectorRepository.ts — SAFE_DIRECT_COPY

```

import type {
  ConnectorAuditAction,
  ConnectorAuditEventDto,
} from '@remotedesk/shared';
import { prisma } from '../db/prisma';

export const connectorRepository = {
  /** Installation map for an org, keyed by connectorKey. */
  async installationsForOrg(
    orgId: string,
  ): Promise<Map<string, { installedAt: Date }>> {
    const rows = await prisma.connectorInstallation.findMany({ where: { orgId } });
    return new Map(rows.map((r) => [r.connectorKey, { installedAt: r.installedAt }]));
  },

  async isInstalled(orgId: string, connectorKey: string): Promise<boolean> {
    const row = await prisma.connectorInstallation.findUnique({
      where: { orgId_connectorKey: { orgId, connectorKey } },
    });
    return !!row;
  },

  async install(
    orgId: string,
    connectorKey: string,
    installedById: string,
    label?: string,
  ): Promise<void> {
    await prisma.connectorInstallation.create({
      data: { orgId, connectorKey, installedById, label: label ?? null },
    });
  },

  async uninstall(orgId: string, connectorKey: string): Promise<number> {
    const { count } = await prisma.connectorInstallation.deleteMany({
      where: { orgId, connectorKey },
    });
    return count;
  },

  async writeAudit(
    orgId: string,
    connectorKey: string,

```

```

    action: ConnectorAuditAction,
    actorUserId: string,
  ): Promise<ConnectorAuditEventDto> {
    const e = await prisma.connectorAuditEvent.create({
      data: { orgId, connectorKey, action, actorUserId },
    });
    return {
      id: e.id,
      orgId: e.orgId,
      connectorKey: e.connectorKey,
      action: e.action as ConnectorAuditAction,
      actorUserId: e.actorUserId,
      createdAt: e.createdAt.toISOString(),
    };
  },
};

```

apps/api/src/services/connector/connectorAudit.ts — SAFE_DIRECT_COPY

```

import type { ConnectorAuditAction } from '@remotedesk/shared';
import { connectorRepository } from '../../repositories/connectorRepository';

/**
 * Thin audit facade so install/uninstall always log through one path. Audit
 * failures must never silently swallow — they throw and roll the request back.
 */
export const connectorAudit = {
  record(
    orgId: string,
    connectorKey: string,
    action: ConnectorAuditAction,
    actorUserId: string,
  ) {
    return connectorRepository.writeAudit(orgId, connectorKey, action, actorUserId);
  },
};

```

apps/api/src/services/connector/connector.service.ts — SAFE_DIRECT_COPY

```

import {
  AppError,
  ErrorCodes,
  getConnectorDefinition,
  isInstallable,
  type ConnectorCatalogEntry,
} from '@remotedesk/shared';
import { buildCatalog, toEntry } from '../../lib/connectorCatalog';
import { connectorRepository } from '../../repositories/connectorRepository';
import { connectorAudit } from './connectorAudit';

export const connectorService = {
  /** Read-only catalog for an org. */
  async catalog(orgId: string): Promise<ConnectorCatalogEntry[]> {
    const installed = await connectorRepository.installationsForOrg(orgId);
    return buildCatalog(installed);
  },
};

```

```

/** Install placeholder: records the installation + audit. No provider calls. */
async install(
  orgId: string,
  connectorKey: string,
  actorUserId: string,
  label?: string,
): Promise<ConnectorCatalogEntry> {
  const def = getConnectorDefinition(connectorKey);
  if (!def) throw new AppError(ErrorCodes.CONNECTOR_NOT_FOUND);
  if (!isInstallable(connectorKey)) {
    throw new AppError(ErrorCodes.CONNECTOR_NOT_INSTALLABLE);
  }
  if (await connectorRepository.isInstalled(orgId, connectorKey)) {
    throw new AppError(ErrorCodes.CONNECTOR_ALREADY_INSTALLED);
  }
  await connectorRepository.install(orgId, connectorKey, actorUserId, label);
  await connectorAudit.record(orgId, connectorKey, 'connector.installed', actorUserId);
  return toEntry(def, { installedAt: new Date() });
},

/** Uninstall placeholder: removes the installation + audit. */
async uninstall(
  orgId: string,
  connectorKey: string,
  actorUserId: string,
): Promise<ConnectorCatalogEntry> {
  const def = getConnectorDefinition(connectorKey);
  if (!def) throw new AppError(ErrorCodes.CONNECTOR_NOT_FOUND);
  const removed = await connectorRepository.uninstall(orgId, connectorKey);
  if (removed === 0) throw new AppError(ErrorCodes.CONNECTOR_NOT_INSTALLED);
  await connectorAudit.record(orgId, connectorKey, 'connector.uninstalled', actorUserId);
  return toEntry(def, undefined);
},
};

```

apps/api/src/routes/connector.routes.ts — SAFE_DIRECT_COPY

```

import { Router } from 'express';
import {
  connectorKeyParamSchema,
  installConnectorSchema,
} from '@remotedesk/shared';
import { requireAuth, type AuthedRequest } from '../middleware/requireAuth.middleware';
import { orgContext, type OrgRequest } from '../middleware/orgContext.middleware';
import { connectorService } from '../services/connector/connector.service';

const h =
  (fn: (req: any, res: any) => Promise<void>) =>
    (req: any, res: any, next: any) =>
      fn(req, res).catch(next);

/**
 * /api/connectors
 * GET / -> read-only catalog for the caller's org
 * POST /:key/install -> install placeholder (audited)
 * DELETE /:key -> uninstall placeholder (audited)
 */

```

```

* orgContext resolves :orgId from the request (header/body) + the caller's role.
*/
export const connectorRoutes = Router()
  .get('/', requireAuth, orgContext, h(async (req, res) => {
    res.json(await connectorService.catalog((req as OrgRequest).orgId));
  })))
  .post('/:key/install', requireAuth, orgContext, h(async (req, res) => {
    const { key } = connectorKeyParamSchema.parse(req.params);
    const { label } = installConnectorSchema.parse(req.body ?? {});
    const r = req as OrgRequest & AuthedRequest;
    res.status(201).json(
      await connectorService.install(r.orgId, key, r.userId, label),
    );
  })))
  .delete('/:key', requireAuth, orgContext, h(async (req, res) => {
    const { key } = connectorKeyParamSchema.parse(req.params);
    const r = req as OrgRequest & AuthedRequest;
    res.json(await connectorService.uninstall(r.orgId, key, r.userId));
  }));

```

apps/api/src/db/seed/connectors.seed.ts — SAFE_DIRECT_COPY

```

import { DEFAULT_CONNECTORS } from '@remotedesk/shared';
import { prisma } from '../prisma';

/**
 * Idempotent seed. Upserts the code-authored catalog into ConnectorDefinition.
 * Safe to run on every boot/deploy. Call from your existing seed entrypoint:
 *   await seedConnectors();
 */
export async function seedConnectors(): Promise<void> {
  for (const def of DEFAULT_CONNECTORS) {
    await prisma.connectorDefinition.upsert({
      where: { key: def.key },
      create: {
        key: def.key,
        name: def.name,
        category: def.category,
        description: def.description,
        availability: def.availability,
        capabilities: [...def.capabilities],
        docsUrl: def.docsUrl ?? null,
      },
      update: {
        name: def.name,
        category: def.category,
        description: def.description,
        availability: def.availability,
        capabilities: [...def.capabilities],
        docsUrl: def.docsUrl ?? null,
      },
    });
  }
}

// Allow running standalone: `tsx src/db/seed/connectors.seed.ts`
if (require.main === module) {

```

```

seedConnectors()
  .then(() => {
    // eslint-disable-next-line no-console
    console.log(`Seeded ${DEFAULT_CONNECTORS.length} connectors`);
    return prisma.$disconnect();
  })
  .catch(async (err) => {
    // eslint-disable-next-line no-console
    console.error(err);
    await prisma.$disconnect();
    process.exit(1);
  });
}

```

apps/api/src/server.ts — REVIEW_REQUIRED (patch)

Mount the router next to the other `/api/*` mounts. Match your existing import style.

```

// ...existing imports...
import { connectorRoutes } from '../routes/connector.routes';

// ...inside createServer(), next to the other app.use('/api/...') mounts:
app.use('/api/connectors', connectorRoutes);

```

Part D — Web: connector catalog page

apps/web/app/dashboard/connectors/connectorApi.ts — SAFE_DIRECT_COPY

```

import type { ConnectorCatalogEntry } from '@remotedesk/shared';
import { api } from '../../../../src/api/apiClient';

/** Wraps the /api/connectors endpoints. No provider calls, no OAuth. */
export const connectorApi = {
  list: () => api.get<ConnectorCatalogEntry[]>('/connectors'),
  install: (key: string, label?: string) =>
    api.post<ConnectorCatalogEntry>(`/connectors/${key}/install`, { label }),
  uninstall: (key: string) =>
    api.delete<ConnectorCatalogEntry>(`/connectors/${key}`),
};

```

apps/web/app/dashboard/connectors/ConnectorCard.tsx — SAFE_DIRECT_COPY

```

'use client';

import React from 'react';
import type { ConnectorCatalogEntry } from '@remotedesk/shared';
import styles from './connectors.module.css';

export function ConnectorCard({
  entry,
  busy,
  onInstall,
  onUninstall,
}: {
  entry: ConnectorCatalogEntry;
  busy: boolean;

```

```

onInstall: (key: string) => void;
onUninstall: (key: string) => void;
}) {
  const comingSoon = entry.status === 'coming_soon';
  const installed = entry.status === 'installed';

  return (
    <div className={styles.card} data-status={entry.status}>
      <div className={styles.cardHead}>
        <span className={styles.name}>{entry.name}</span>
        <span className={styles.badge} data-status={entry.status}>
          {comingSoon ? 'Coming soon' : installed ? 'Installed' : 'Available'}
        </span>
      </div>
      <p className={styles.desc}>{entry.description}</p>
      <div className={styles.caps}>
        {entry.capabilities.map((c) => (
          <span key={c} className={styles.cap}>>{c}</span>
        ))}
      </div>
      <div className={styles.actions}>
        {comingSoon ? (
          <button type="button" disabled className={styles.btnGhost}>
            Coming soon
          </button>
        ) : installed ? (
          <button
            type="button"
            disabled={busy}
            className={styles.btnDanger}
            onClick={() => onUninstall(entry.key)}
          >
            {busy ? 'Working...' : 'Uninstall'}
          </button>
        ) : (
          <button
            type="button"
            disabled={busy}
            className={styles.btnPrimary}
            onClick={() => onInstall(entry.key)}
          >
            {busy ? 'Working...' : 'Install'}
          </button>
        )}
        {entry.docsUrl && (
          <a className={styles.docsLink} href={entry.docsUrl} target="_blank" rel="noreferrer">
            Docs
          </a>
        )}
      </div>
    </div>
  );
}

```

```

'use client';

import React, { useCallback, useEffect, useState } from 'react';
import type { ConnectorCatalogEntry } from '@remotedesk/shared';
import { connectorApi } from './connectorApi';
import { ConnectorCard } from './ConnectorCard';
import styles from './connectors.module.css';

export default function ConnectorsPage() {
  const [entries, setEntries] = useState<ConnectorCatalogEntry[]>([]);
  const [loading, setLoading] = useState(true);
  const [busyKey, setBusyKey] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);

  const load = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      setEntries(await connectorApi.list());
    } catch {
      setError('Could not load connectors.');
```



```

<header className={styles.header}>
  <h1>Connectors</h1>
  <p className={styles.sub}>
    Connect RemoteDesk to your team's tools. OAuth setup arrives in a
    later release.
  </p>
</header>

{error && <div className={styles.error}><div>{error}</div>
{loading ? (
  <div className={styles.loading}>Loading catalog...</div>
) : (
  <div className={styles.grid}>
    {entries.map((entry) => (
      <ConnectorCard
        key={entry.key}
        entry={entry}
        busy={busyKey === entry.key}
        onInstall={install}
        onUninstall={uninstall}
      />
    ))}
  </div>
)}
</main>
);
}

```

[apps/web/app/dashboard/connectors/connectors.module.css](#) — SAFE_DIRECT_COPY

```

.page { padding: 24px; max-width: 1080px; margin: 0 auto; }
.header h1 { margin: 0 0 4px; font-size: 22px; }
.sub { color: var(--muted, #6b7280); margin: 0 0 20px; }
.error { background: #fef2f2; color: #991b1b; padding: 10px 14px; border-radius: 8px; margin-bottom: 16px; }
.loading { color: var(--muted, #6b7280); padding: 32px 0; }
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 16px; }
.card { border: 1px solid var(--border, #e5e7eb); border-radius: 12px; padding: 16px; display: flex; flex-direction:
column; gap: 10px; background: var(--surface, #fff); }
.card[data-status='coming_soon'] { opacity: 0.75; }
.cardHead { display: flex; align-items: center; justify-content: space-between; }
.name { font-weight: 600; font-size: 15px; }
.badge { font-size: 11px; padding: 2px 8px; border-radius: 999px; background: #eef2ff; color: #3730a3; }
.badge[data-status='installed'] { background: #ecfdf5; color: #065f46; }
.badge[data-status='coming_soon'] { background: #f3f4f6; color: #6b7280; }
.desc { font-size: 13px; color: var(--muted, #6b7280); margin: 0; min-height: 34px; }
.caps { display: flex; flex-wrap: wrap; gap: 6px; }
.cap { font-size: 11px; padding: 2px 6px; border-radius: 6px; background: #f3f4f6; color: #374151; }
.actions { display: flex; align-items: center; gap: 10px; margin-top: auto; }
.btnPrimary { background: #4f46e5; color: #fff; border: none; padding: 8px 14px; border-radius: 8px; cursor: pointer; }
.btnDanger { background: #fff; color: #b91c1c; border: 1px solid #fecaca; padding: 8px 14px; border-radius: 8px; cursor:
pointer; }
.btnGhost { background: #f3f4f6; color: #9ca3af; border: none; padding: 8px 14px; border-radius: 8px; cursor: not-
allowed; }
.btnPrimary:disabled, .btnDanger:disabled { opacity: 0.6; cursor: default; }
.docsLink { font-size: 12px; color: #4f46e5; text-decoration: none; }

```

[apps/web/app/dashboard/page.tsx](#) — REVIEW_REQUIRED (patch)

Add a link/card into your existing dashboard grid. Adjust to your component set.

```
// ...existing imports...
import Link from 'next/link';

// ...inside the dashboard nav/grid, alongside the other section links:
<Link href="/dashboard/connectors" className="dashboard-card">
  <h3>Connectors</h3>
  <p>Slack, Teams, Jira, Zendesk, S3 and more.</p>
</Link>
```

Part E — Desktop: device policy enforcement

apps/desktop/src/renderer/src/services/devicePolicy.ts — SAFE_DIRECT_COPY

```
import {
  failClosedPolicy,
  isCapabilityAllowed,
  isDevicePolicy,
  type DevicePolicy,
  type PolicyCapability,
} from '@remotedesk/shared';

/**
 * In-memory policy store for the desktop client. Starts FAIL-CLOSED for the
 * device and only relaxes once a valid policy is fetched. Any malformed payload
 * is rejected and the store stays/returns to fail-closed.
 */
export class DevicePolicyStore {
  private policy: DevicePolicy;

  constructor(private readonly deviceId: string) {
    this.policy = failClosedPolicy(deviceId);
  }

  get current(): DevicePolicy {
    return this.policy;
  }

  /** True only if a real (non-fail-closed) policy has been loaded. */
  get isHydrated(): boolean {
    return this.policy.version > 0;
  }

  /**
   * Accept a fetched payload. Returns true if applied. On any invalid/stale
   * payload we DROP to fail-closed (do not keep a possibly-revoked allow state
   * around longer than necessary).
   */
  apply(payload: unknown): boolean {
    if (!isDevicePolicy(payload) || payload.deviceId !== this.deviceId) {
      this.failClosed();
      return false;
    }

    // Ignore strictly older versions, but never let a stale fetch widen access.
```

```

    if (payload.version < this.policy.version) return false;
    this.policy = payload;
    return true;
  }

  /** Force everything denied (called on fetch failure / API unavailable). */
  failClosed(): void {
    this.policy = failClosedPolicy(this.deviceId);
  }

  allows(capability: PolicyCapability): boolean {
    return isCapabilityAllowed(this.policy, capability);
  }
}

```

apps/desktop/src/renderer/src/services/devicePolicyPoller.ts — SAFE_DIRECT_COPY

```

import { type DevicePolicyStore } from './devicePolicy';

/** Transport injected so tests don't touch the network. */
export type PolicyFetch = (deviceId: string) => Promise<unknown>;

export interface PollerOptions {
  intervalMs?: number;
  /** Called whenever the effective policy changes (e.g. to re-gate the UI). */
  onChange?: () => void;
}

/**
 * Polls the API for the device policy on an interval. On ANY failure (network,
 * non-2xx, malformed body) it forces the store fail-closed, so a device that
 * loses contact with the control plane immediately loses sensitive capabilities.
 */
export class DevicePolicyPoller {
  private timer: ReturnType<typeof setInterval> | null = null;
  private readonly intervalMs: number;

  constructor(
    private readonly deviceId: string,
    private readonly store: DevicePolicyStore,
    private readonly fetchPolicy: PolicyFetch,
    private readonly opts: PollerOptions = {},
  ) {
    this.intervalMs = opts.intervalMs ?? 30_000;
  }

  async start(): Promise<void> {
    await this.tick();
    this.timer = setInterval(() => void this.tick(), this.intervalMs);
  }

  stop(): void {
    if (this.timer) clearInterval(this.timer);
    this.timer = null;
  }

  /** One poll cycle. Exposed for tests + manual refresh. */

```

```

async tick(): Promise<void> {
  const before = this.store.current.version;
  try {
    const payload = await this.fetchPolicy(this.deviceId);
    this.store.apply(payload);
  } catch {
    this.store.failClosed();
  }
  if (this.store.current.version !== before) this.opts.onChange?.();
}
}

/** Default transport over fetch. Throws on non-2xx so the poller fails closed. */
export function httpPolicyFetch(baseUrl: string, authToken: string): PolicyFetch {
  return async (deviceId: string) => {
    const res = await fetch(`${baseUrl}/api/devices/${deviceId}/policy`, {
      headers: { authorization: `Bearer ${authToken}` },
    });
    if (!res.ok) throw new Error(`policy fetch failed: ${res.status}`);
    return res.json();
  };
}

```

apps/desktop/src/renderer/src/services/devicePolicyGuards.ts — SAFE_DIRECT_COPY

```

import { type DevicePolicyStore } from './devicePolicy';
import { type PolicyCapability } from '@remotedesk/shared';

/**
 * Pure gate functions the session pipeline calls before performing a sensitive
 * action. All default to DENY: if the store is anything but explicitly allowed,
 * the action is blocked.
 */
export class DevicePolicyGuards {
  constructor(private readonly store: DevicePolicyStore) {}

  private gate(capability: PolicyCapability): boolean {
    return this.store.allows(capability);
  }

  canSendRemoteInput(): boolean {
    return this.gate('remoteInput');
  }

  canSyncClipboard(): boolean {
    return this.gate('clipboard');
  }

  canTransferFiles(): boolean {
    return this.gate('fileTransfer');
  }

  canCaptureAudio(): boolean {
    return this.gate('audio');
  }

  /** Throwing variant for call sites that must hard-stop. */

```

```

assert(capability: PolicyCapability): void {
  if (!this.store.allows(capability)) {
    throw new Error(`device policy denies "${capability}"`);
  }
}
}
}

```

apps/desktop/src/renderer/src/session/sessionSetup.ts — REVIEW_REQUIRED (patch)

Wire the guards into your existing session setup. Names below mirror the desktop session engine from earlier batches (input sender, clipboard sync, file transfer). Merge into the real handlers — do not replace your session bootstrap.

```

// ...existing imports...
import { DevicePolicyStore } from '../services/devicePolicy';
import { DevicePolicyPoller, httpPolicyFetch } from '../services/devicePolicyPoller';
import { DevicePolicyGuards } from '../services/devicePolicyGuards';

// --- during session bootstrap, after you know deviceId + auth token + apiBase ---
const policyStore = new DevicePolicyStore(deviceId);
const policyGuards = new DevicePolicyGuards(policyStore);
const policyPoller = new DevicePolicyPoller(
  deviceId,
  policyStore,
  httpPolicyFetch(apiBase, authToken),
  { onChange: () => refreshSessionCapabilityUI() }, // your existing UI refresh
);
await policyPoller.start();

// --- gate the existing remote-input handler ---
inputChannel.on('input', (event) => {
  if (!policyGuards.canSendRemoteInput()) return; // fail closed: drop the event
  applyRemoteInput(event); // existing behavior
});

// --- gate clipboard sync ---
clipboardSync.onIncoming((payload) => {
  if (!policyGuards.canSyncClipboard()) return;
  writeClipboard(payload); // existing behavior
});

// --- gate file transfer ---
fileTransfer.onTransferRequest((req) => {
  if (!policyGuards.canTransferFiles()) {
    fileTransfer.reject(req.id, 'blocked_by_policy');
    return;
  }
  fileTransfer.accept(req.id); // existing behavior
});

// --- on teardown ---
session.onClose(() => policyPoller.stop());

```

All tests import only real shared/copied files. No REVIEW_REQUIRED-only imports.

tests/connectors/catalog.test.ts — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import {
  DEFAULT_CONNECTORS,
  getConnectorDefinition,
  resolveStatus,
} from '@remotedesk/shared';
import { buildCatalog } from '../../apps/api/src/lib/connectorCatalog';

describe('connector catalog', () => {
  it('ships the six default connectors with unique keys', () => {
    const keys = DEFAULT_CONNECTORS.map((c) => c.key);
    expect(keys).toEqual(['slack', 'teams', 'jira', 'zendesk', 's3', 'siem-webhook']);
    expect(new Set(keys).size).toBe(keys.length);
  });

  it('marks the SIEM webhook as coming soon', () => {
    expect(getConnectorDefinition('siem-webhook')?.availability).toBe('coming_soon');
  });

  it('builds an available entry when not installed', () => {
    const catalog = buildCatalog(new Map());
    const slack = catalog.find((e) => e.key === 'slack')!;
    expect(slack.installed).toBe(false);
    expect(slack.status).toBe('available');
    expect(slack.installedAt).toBeNull();
  });

  it('builds an installed entry when present in the install map', () => {
    const now = new Date();
    const catalog = buildCatalog(new Map([[ 'jira', { installedAt: now } ]]));
    const jira = catalog.find((e) => e.key === 'jira')!;
    expect(jira.installed).toBe(true);
    expect(jira.status).toBe('installed');
    expect(jira.installedAt).toBe(now.toISOString());
  });

  it('keeps coming_soon status even if somehow installed', () => {
    expect(resolveStatus({ availability: 'coming_soon' }, true)).toBe('coming_soon');
  });
});
```

tests/connectors/safety.test.ts — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import {
  assertNoSecretFields,
  isInstallable,
  normalizeConnectorKey,
  resolveStatus,
} from '@remotedesk/shared';

describe('connector safety', () => {
  it('rejects payloads carrying secret-looking fields', () => {
```

```

    expect(() => assertNoSecretFields({ name: 'ok' })).not.toThrow();
    expect(() => assertNoSecretFields({ client_secret: 'x' })).toThrow();
    expect(() => assertNoSecretFields({ apiKey: 'x' })).toThrow();
  });

  it('only allows installing available connectors', () => {
    expect(isInstallable('slack')).toBe(true);
    expect(isInstallable('siem-webhook')).toBe(false); // coming_soon
    expect(isInstallable('does-not-exist')).toBe(false);
  });

  it('normalizes keys and rejects unknown ones', () => {
    expect(normalizeConnectorKey(' SLACK ')).toBe('slack');
    expect(normalizeConnectorKey(' nope')).toBeNull();
  });

  it('resolves status from availability + install state', () => {
    expect(resolveStatus({ availability: 'available' }, false)).toBe('available');
    expect(resolveStatus({ availability: 'available' }, true)).toBe('installed');
  });
});

```

tests/device-policy/policy-enforcement.test.ts — SAFE_DIRECT_COPY

```

import { describe, it, expect, vi } from 'vitest';
import { failClosedPolicy, type DevicePolicy } from '@remotedesk/shared';
import { DevicePolicyStore } from '../../apps/desktop/src/renderer/src/services/devicePolicy';
import { DevicePolicyGuards } from '../../apps/desktop/src/renderer/src/services/devicePolicyGuards';
import { DevicePolicyPoller } from '../../apps/desktop/src/renderer/src/services/devicePolicyPoller';

const DEVICE = 'dev-1';

function allowAll(version = 1): DevicePolicy {
  return {
    deviceId: DEVICE,
    allowRemoteInput: true,
    allowClipboard: true,
    allowFileTransfer: true,
    allowAudio: true,
    maxSessionMinutes: null,
    version,
    updatedAt: new Date().toISOString(),
  };
}

describe('device policy enforcement', () => {
  it('starts fully fail-closed before any fetch', () => {
    const store = new DevicePolicyStore(DEVICE);
    const guards = new DevicePolicyGuards(store);
    expect(store.isHydrated).toBe(false);
    expect(guards.canSendRemoteInput()).toBe(false);
    expect(guards.canSyncClipboard()).toBe(false);
    expect(guards.canTransferFiles()).toBe(false);
    expect(guards.canCaptureAudio()).toBe(false);
  });

  it('applies a valid policy and relaxes gates', () => {

```

```

    const store = new DevicePolicyStore(DEVICE);
    const guards = new DevicePolicyGuards(store);
    expect(store.apply(allowAll())).toBe(true);
    expect(guards.canSendRemoteInput()).toBe(true);
    expect(guards.canTransferFiles()).toBe(true);
  });

  it('rejects malformed payloads and stays fail-closed', () => {
    const store = new DevicePolicyStore(DEVICE);
    expect(store.apply({ deviceId: DEVICE, allowRemoteInput: 'yes' })).toBe(false);
    expect(store.allows('remoteInput')).toBe(false);
  });

  it('rejects a policy for a different device', () => {
    const store = new DevicePolicyStore(DEVICE);
    expect(store.apply({ ...allowAll(), deviceId: 'other' })).toBe(false);
    expect(store.allows('clipboard')).toBe(false);
  });

  it('poller forces fail-closed when the fetch throws', async () => {
    const store = new DevicePolicyStore(DEVICE);
    store.apply(allowAll()); // start allowed
    const poller = new DevicePolicyPoller(
      DEVICE,
      store,
      vi.fn().mockRejectedValue(new Error('offline')),
      { intervalMs: 999_999 },
    );
    await poller.tick();
    expect(store.allows('remoteInput')).toBe(false);
    expect(store.current.version).toBe(failClosedPolicy(DEVICE).version);
  });

  it('poller applies a fresh policy on success', async () => {
    const store = new DevicePolicyStore(DEVICE);
    const onChange = vi.fn();
    const poller = new DevicePolicyPoller(
      DEVICE,
      store,
      vi.fn().mockResolvedValue(allowAll(2)),
      { intervalMs: 999_999, onChange },
    );
    await poller.tick();
    expect(store.allows('fileTransfer')).toBe(true);
    expect(onChange).toHaveBeenCalledTimes(1);
  });
});

```

Part G — Docs

[docs/connector-catalog.md](#) — SAFE_DIRECT_COPY

Connector Catalog

A read-only catalog of third-party connectors with install/uninstall placeholders.

****Pack 11 ships no OAuth, no secrets, and makes no real provider calls.****

Source of truth

All connector definitions live in `packages/shared/src/connectors/catalog.ts` (`DEFAULT\_CONNECTORS`). The API seed upserts them into `ConnectorDefinition`, the API service falls back to them, and the web UI + tests assert against the same list. Editing a connector happens in exactly one place.

Connectors

Key	Name	Category	Status
---	---	---	---
slack	Slack	chat	available
teams	Microsoft Teams	chat	available
jira	Jira	ticketing	available
zendesk	Zendesk	ticketing	available
s3	Amazon S3	storage	available
siem-webhook	SIEM Webhook	security	coming soon

API

- `GET /api/connectors` — catalog for the caller's org (definition + install state).
- `POST /api/connectors/:key/install` — install placeholder; writes a `connector.installed` audit event.
- `DELETE /api/connectors/:key` — uninstall placeholder; writes a `connector.uninstalled` audit event.

Install is rejected for unknown keys, `coming\_soon` connectors, and connectors already installed for the org. Uninstall is rejected when nothing is installed.

Audit

Every install/uninstall writes a `ConnectorAuditEvent` (org, connector, action, actor, timestamp) before returning. Audit write failures roll the request back.

Status model

`status` is derived: `coming\_soon` always wins; otherwise `installed` vs `available`. The same `resolveStatus` helper runs on server and client.

docs/device-policy-enforcement.md — SAFE_DIRECT_COPY

Device Policy Enforcement (Desktop)

The desktop client enforces a per-device policy over the three sensitive session capabilities — **remote input**, **clipboard sync**, **file transfer** (plus audio) — and **fails closed**.

Fail-closed by default

`DevicePolicyStore` starts at `failClosedPolicy(deviceId)`: every capability denied, `version = 0`. Capabilities are only enabled after a **valid** policy is fetched. Any of these drop the device back to fully denied:

- API unreachable / non-2xx / network error
- malformed policy payload
- a policy whose `deviceId` doesn't match this device

Components

- `services/devicePolicy.ts` — in-memory `DevicePolicyStore` (apply/failClosed/allows).
- `services/devicePolicyPoller.ts` — interval poller; on **any** fetch failure it forces fail-closed. Transport is injected (`httpPolicyFetch` for prod, mocks in tests).
- `services/devicePolicyGuards.ts` — pure gate functions the session pipeline calls before sending input, syncing clipboard, or transferring files.

Wiring

`session/sessionSetup.ts` (patch) constructs the store + guards + poller during session bootstrap, gates the existing input/clipboard/file-transfer handlers on the guards, and stops the poller on teardown.

Contract

Policy shape is `DevicePolicy` in `packages/shared/src/policy/devicePolicy.ts`. The poller expects `GET /api/devices/:deviceId/policy` to return that shape. `version` is monotonic; a strictly older fetch never widens access.

Why client-side gating

The desktop client is the last line before the OS performs the action, so gating here guarantees a revoked or unreachable policy stops input/clipboard/file transfer even if a server-side check is bypassed. Server-side enforcement remains a follow-up.

Part H — Handoff meta (doNotMerge)

`generated-remotedesk-pack11-merge-summary.md` — doNotMerge

RemoteDesk Pack 11 — Merge Summary

****Pack:**** Connector Catalog + Device Policy Enforcement

****Files:**** 35 (29 SAFE_DIRECT_COPY, 6 REVIEW_REQUIRED, 3 of which are doNotMerge meta)

Build order

1. `packages/shared` — connectors/* + policy/* (then patch shared `index.ts` + `errors/errorCodes.ts`), `pnpm --filter @remotedesk/shared build`.
2. `apps/api` — patch `schema.prisma`, run the `0011\_connector\_catalog` migration + `prisma generate`, drop in lib/repo/service/routes/seed, patch `server.ts`, run `seedConnectors()`.
3. `apps/web` — drop in `dashboard/connectors/\*`, patch the dashboard page link.
4. `apps/desktop` — drop in the three policy services, patch `session/sessionSetup.ts`.
5. Tests — `pnpm -r test`.

REVIEW_REQUIRED (hand-merge)

- `packages/shared/src/index.ts` — add two barrels.
- `packages/shared/src/errors/errorCodes.ts` — add 4 connector codes.
- `apps/api/prisma/schema.prisma` — add 3 models + indexes.
- `apps/api/src/server.ts` — mount `/api/connectors`.
- `apps/web/app/dashboard/page.tsx` — add catalog link.
- `apps/desktop/src/renderer/src/session/sessionSetup.ts` — wire guards into existing handlers.

Known limitations

- Install/uninstall are placeholders: they persist + audit only. No OAuth, no provider calls.
- Device-policy enforcement is client-side; the `GET /api/devices/:id/policy` endpoint is assumed from the existing device API. If absent, add it (returns `DevicePolicy`) before the poller works in prod.
- No server-side capability enforcement yet (follow-up pack).
- `apiClient` (`api.get/post/delete`) and `requireAuth`/`orgContext` middleware are assumed from prior batches.

Verify

- `pnpm -r typecheck && pnpm -r test`
- Manual: open `/dashboard/connectors`, install Slack, confirm a `ConnectorAuditEvent` row, uninstall.
- Manual: kill the API, confirm the desktop session blocks remote input/clipboard/file transfer.

```
# RemoteDesk Pack 11 — Code Review Notes
```

Strengths

- Single source of truth: `DEFAULT\_CONNECTORS` drives seed, service, UI, and tests.
- Status logic (`resolveStatus`) and capability checks (`isCapabilityAllowed`) are pure and shared, so server and client can't drift.
- Device policy is fail-closed at every failure mode (network, malformed, wrong device, never-fetched).
- Tests import only real generated/copied files — no REVIEW_REQUIRED-only imports.

Reviewer checklist

- [] `schema.prisma` patch: confirm the org relation strategy matches existing convention (loose `orgId` used here, matching prior batches).
- [] `apiClient` exposes `delete<T>()`. If not, add it or switch uninstall to `post`.
- [] `orgContext` resolves `orgId` for these routes (header/body). Confirm the client sends it.
- [] Desktop poller interval (30s default) acceptable vs your heartbeat cadence.
- [] `GET /api/devices/:id/policy` exists and returns the `DevicePolicy` shape.

Security

- No secrets in any connector payload; `assertNoSecretFields` guards against regressions.
- Coming-soon connectors are non-installable on both server (`isInstallable`) and client (button disabled).
- Audit is written before the response returns; failures roll back the install/uninstall.

```
{
  "pack": "pack11",
  "title": "Connector Catalog + Device Policy Enforcement",
  "actualFileCount": 35,
  "safeDirectCopy": [
    "packages/shared/src/connectors/types.ts",
    "packages/shared/src/connectors/catalog.ts",
    "packages/shared/src/connectors/safety.ts",
    "packages/shared/src/connectors/schema.ts",
    "packages/shared/src/connectors/index.ts",
    "packages/shared/src/policy/devicePolicy.ts",
    "packages/shared/src/policy/index.ts",
    "apps/api/prisma/migrations/0011_connector_catalog/migration.sql",
    "apps/api/src/lib/connectorCatalog.ts",
    "apps/api/src/repositories/connectorRepository.ts",
    "apps/api/src/services/connector/connector.service.ts",
    "apps/api/src/services/connector/connectorAudit.ts",
    "apps/api/src/routes/connector.routes.ts",
    "apps/api/src/db/seed/connectors.seed.ts",
    "apps/web/app/dashboard/connectors/page.tsx",
    "apps/web/app/dashboard/connectors/connectorApi.ts",
    "apps/web/app/dashboard/connectors/ConnectorCard.tsx",
    "apps/web/app/dashboard/connectors/connectors.module.css",
    "apps/desktop/src/renderer/src/services/devicePolicy.ts",
    "apps/desktop/src/renderer/src/services/devicePolicyPoller.ts",
    "apps/desktop/src/renderer/src/services/devicePolicyGuards.ts",
    "tests/connectors/catalog.test.ts",
    "tests/connectors/safety.test.ts",
    "tests/device-policy/policy-enforcement.test.ts",
    "docs/connector-catalog.md",
    "docs/device-policy-enforcement.md"
  ]
}
```

```

],
"reviewRequired": [
  "packages/shared/src/errors/errorCodes.ts",
  "packages/shared/src/index.ts",
  "apps/api/prisma/schema.prisma",
  "apps/api/src/server.ts",
  "apps/web/app/dashboard/page.tsx",
  "apps/desktop/src/renderer/src/session/sessionSetup.ts"
],
"doNotMerge": [
  "generated-remotedesk-pack11-merge-summary.md",
  "generated-remotedesk-pack11-code-review.md",
  "generated-remotedesk-pack11-manifest.json"
],
"dependenciesRequired": {
  "new": [],
  "existing": ["zod", "@prisma/client", "express", "next", "react", "vitest"],
  "assumedFromPriorBatches": [
    "@remotedesk/shared AppError + ErrorCodes",
    "apps/api db/prisma client",
    "apps/api middleware: requireAuth, orgContext",
    "apps/web src/api/apiClient (api.get/post/delete)",
    "apps/desktop session engine handlers (input/clipboard/fileTransfer)",
    "GET /api/devices/:id/policy returning DevicePolicy"
  ]
},
"knownLimitations": [
  "Install/uninstall are placeholders: persist + audit only, no OAuth, no provider calls.",
  "Device policy enforcement is client-side; server-side capability enforcement is a follow-up.",
  "Poller depends on GET /api/devices/:id/policy; add it if not already present.",
  "SIEM webhook connector is coming_soon and intentionally non-installable."
]
}

```

Pack 11 done: 35 files. Read-only connector catalog (6 connectors, one shared source of truth), install/uninstall placeholders with mandatory audit logging and zero secrets, a Next.js catalog page with available/installed/coming-soon cards, and desktop device-policy enforcement that fails **closed** for remote input, clipboard, and file transfer the moment the API goes dark.

The clean bit: `DEFAULT_CONNECTORS` and the status/capability helpers live once in `@remotedesk/shared`, so seed, API, web, and tests can never disagree. The desktop never starts in an "allowed" state: it boots denied and only relaxes on a verified, device-matched, current policy.

Next number = Pack 12 (connector OAuth + real provider calls), or server-side device-policy enforcement. Bolo.